Šepet

Varno sporočanje s šifriranjem od konca do konca

Prijava teme projekta · Računalniške storitve v oblaku 2026/27

1 Osnovni podatki

Naslov projekta	Šepet — varno sporočanje s šifriranjem od konca do konca
Skupina	Projektna ekipa 13, ime skupine: Ključarji
Člani	Tibor Koderman, Amar Mehić, Eva Vidic
Repozitoriji	https://gitlab.koderverse.com/fri-mag/rso (en repozitorij za vsako mikrostoritev)

2 Kratek opis projekta

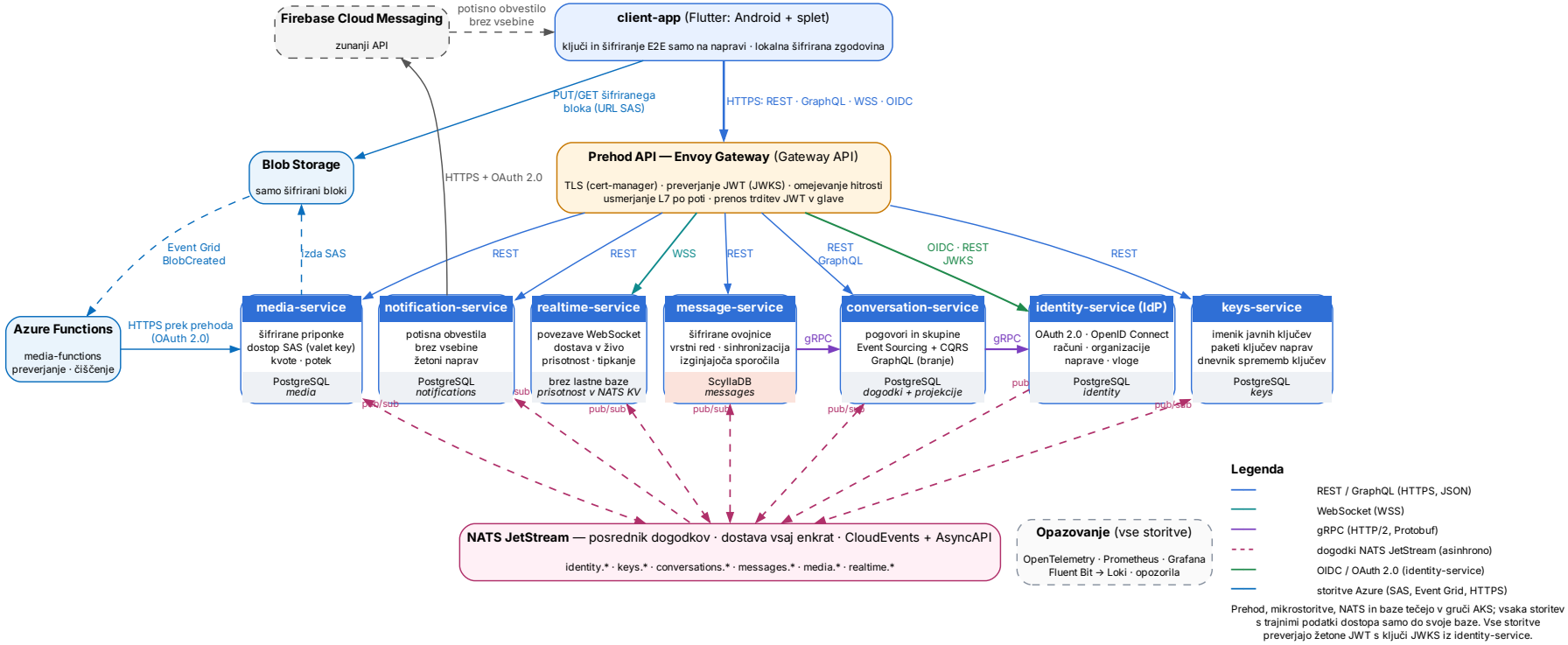
Pri večini aplikacij za klepet moramo ponudniku zaupati, da naših sporočil ne bo bral ali jih predal tretjim osebam. Telegram na primer klepetov privzeto ne šifrira od konca do konca, hkrati pa so vse pogostejše zahteve po dostopu oblasti do zasebnih sporočil. Razvili bomo aplikacijo Šepet za sporočanje ena na ena in v skupinah, ki podpira slike in oblikovano besedilo. Sporočila in priponke se šifrirajo na napravi pošiljatelja s protokolom MLS (RFC 9420) in dešifrirajo šele na napravah prejemnikov, zato strežnik hrani in posreduje samo šifrirane podatke in vsebine ne more prebrati. Aplikacijo bomo razvili kot sistem mikrostoritev in jo namestili v oblak Azure (Kubernetes).

3 Ogrodje in razvojno okolje

- **Uporabniški vmesnik:** Flutter (aplikacija za Android in splet).
- **Šifriranje:** protokol MLS s knjižnico OpenMLS; kot rezervna možnost Signal Protocol.
- **Mikrostoritve:** .NET 10 (ASP.NET Core) po načelih Clean Architecture.
- **Podatkovne baze:** PostgreSQL, ScyllaDB za sporočila, Azure Blob Storage za šifrirane slike.
- **Komunikacija:** REST, GraphQL, gRPC, WebSocket in NATS JetStream za dogodke.
- **Avtentikacija:** lasten ponudnik identitet v .NET (OAuth 2.0, OpenID Connect).
- **Infrastruktura:** Docker, Kubernetes (Azure AKS), Helm, Envoy Gateway kot prehod API, Terraform.
- **CI/CD:** GitLab CI.
- **Spremljanje:** Prometheus, Grafana, Loki.
- **Ostalo:** Azure Functions, Firebase Cloud Messaging za potisna obvestila.
- **Razvojna orodja:** JetBrains Rider, VS Code, Android Studio, Docker Compose in kind za lokalni razvoj, Git.

4 Shema arhitekture

Aplikacija dostopa do sistema prek prehoda API, ki preverja žetone in usmerja zahteve do mikrostoritev (REST, GraphQL, WebSocket). Med storitvami uporabljamo gRPC, kadar potrebujemo takojšen odgovor (npr. preverjanje članstva pri pošiljanju sporočila), za obveščanje o dogodkih (npr. novo sporočilo) pa posrednika sporočil NATS JetStream. Vsaka storitev, ki hrani podatke, ima svojo podatkovno bazo. Shema je na sliki 1 (naslednja stran).



Slika 1: Začetna shema arhitekture s komunikacijskimi protokoli.

5 Mikrostoritve in njihove funkcionalnosti

Mikrostoritev	Funkcionalnosti	Baza	Odgovorna oseba
identity-service	registracija in prijava (OAuth 2.0, OpenID Connect), organizacije, vloge, naprave uporabnika	PostgreSQL	Tibor
keys-service	javni ključi naprav, paketi ključev za vzpostavitev šifriranja, opozorilo ob spremembi ključa	PostgreSQL	Tibor
conversation-service	pogovori ena na ena in skupine, dodajanje in odstranjevanje članov; Event Sourcing in CQRS, GraphQL	PostgreSQL	Amar
message-service	shranjevanje in vrstni red šifriranih sporočil, sinhronizacija naprav, izginjajoča sporočila	ScyllaDB	Amar
realtime-service	povezave WebSocket, dostava sporočil v živo, prisotnost in tipkanje	—	Amar
media-service	nalaganje in prenos šifriranih slik (Azure Blob Storage), omejitve velikosti	PostgreSQL	Eva
notification-service	potisna obvestila brez vsebine prek Firebase Cloud Messaging	PostgreSQL	Eva

Poleg mikrostoritev bomo razvili še aplikacijo v Flutterju in funkcijo v Azure Functions, ki preveri naložene slike in briše potekle (Eva).

6 Primeri uporabe

- registracija in prijava v aplikacijo
- iskanje uporabnika in začetek pogovora
- pošiljanje in prejemanje šifriranih sporočil z oblikovanim besedilom
- pošiljanje slik
- ustvarjanje skupine ter dodajanje in odstranjevanje članov
- potisno obvestilo, ko aplikacija ni odprta
- preverjanje varnostne številke stika s kodo QR
- izginjajoča sporočila
- upravljanje organizacije in vlog uporabnikov (skrbnik)
- preklic izgubljene naprave

Kompleksnejši primer uporabe — pošiljanje slike v skupinski pogovor:

1. Aplikacija sliko šifrira z naključnim ključem in pri **media-service** zahteva povezavo za nalaganje.
2. Šifrirana slika se naloži v Azure Blob Storage; **Azure Function** jo preveri in o tem obvesti media-service.
3. Aplikacija pošlje šifrirano sporočilo s ključem slike storitvi **message-service**, ki prek gRPC pri **conversation-service** preveri, ali je pošiljatelj član skupine.
4. **message-service** sporočilo shrani v ScyllaDB in objavi dogodek v NATS JetStream.
5. **realtime-service** sporočilo prek WebSocketa dostavi povezanim članom, **notification-service** pa ostalim pošlje potisno obvestilo.
6. Prejemniki sporočilo dešifrirajo, prenesejo šifrirano sliko in jo dešifrirajo na svoji napravi.